Zadanie č. 2b – Klastrovanie

Ruslan Gainetdinov, ID: 127150

November 2025

Abstrakt

Cieľom projektu bolo vygenerovať 40 020 náhodných bodov v dvojrozmernom priestore, ktoré tvoria k prirodzených zhlukov, a rozdeliť ich do k z zhlukov pomocou troch rôznych algoritmov: k-means s centroidmi, k-medoids a divízneho zhlukovania s centroidmi. Výsledky boli vizualizované a porovnané.

1 Generovanie dát

Na začiatku bolo náhodne vygenerovaných 20 počiatočných bodov v intervale od -5000 do 5000 pre obe súradnice. Tieto body slúžia ako centrá budúcich zhlukov. Následne bolo pridaných ďalších 40 000 bodov tak, že každý nový bod vznikol náhodným posunom existujúceho bodu o maximálne ± 100 v oboch smeroch, pričom posun bol obmedzený, aby bod neprekročil hranice priestoru. Týmto spôsobom vzniklo 40 020 bodov v 20 jasne oddelených zhlukoch.

2 Použité algoritmy

2.1 k-means s centroidmi

Algoritmus k-means je iteratívna metóda, ktorá rozdeľuje body do k zhlukov minimalizáciou súčtu štvorcov vzdialeností od centroidov. Pôvodná verzia s náhodným výberom počiatočných centroidov často viedla k zlým výsledkom – niektoré klastre mali priemernú vzdialenosť vyše 600, čo nespĺňalo podmienku zadania.

Preto bola implementovaná pokročilá inicializácia k-means++, ktorá vyberá počiatočné centroidy inteligentne: prvý bod náhodne, každý ďalší s pravdepodobnosťou úmernou štvorcu jeho vzdialenosti od najbližšieho už vybraného centroidu. Tento prístup zaručuje rovnomerné rozloženie centroidov. Algoritmus bol spustený 10-krát a vybraný bol výsledok s najnižšou celkovou chybou. Vďaka tomu dosiahol priemernú vzdialenosť vo všetkých klastroch a vždy splnil podmienku zadania.

2.2 k-medoids

k-medoids je robustnejšia alternatíva k-means, kde stred klastra (medoid) musí byť skutočný bod z dát. Algoritmus začína rovnakou k-means++ inicializáciou. V každej iterácii sa pre každý klaster skúšajú náhodne vybrané body ako potenciálne nové medoidy. Ak niektorý z nich zníži celkovú chybu zhlukovania, okamžite sa použije. Proces sa opakuje, kým sa už nedá nič zlepšiť.

Výhodou je väčšia odolnosť voči odľahlým bodom. Nevýhodou je vyššia výpočtová náročnosť. Napriek tomu algoritmus úspešne rozdelil body do 20 zhlukov.

2.3 Divízne zhlukovanie s centroidmi

Tento algoritmus pracuje opačným spôsobom ako aglomeratívne zhlukovanie – začína jedným veľkým klastrom obsahujúcim všetky body. V každom kroku vyberie klaster s najväčšou priemernou vzdialenosťou bodov od jeho centroidu a rozdelí ho na dva podklastre pomocou k-means s dvoma centroidmi. Tento proces sa opakuje, kým sa nedosiahne požadovaný počet 20 zhlukov.

Výhodou je, že prirodzene vytvára hierarchickú štruktúru a dobre funguje na guľovitých zhlukoch.

3 Parametre algoritmov a ich zdôvodnenie

Pri implementácii algoritmov bolo potrebné zvoliť viacero parametrov. Ich hodnoty neboli vybrané náhodne, ale na základe experimentov a požiadaviek zadania (dosiahnutie `Úspech: True` pri zachovaní rozumného času behu).

3.1 k-means s centroidmi

- **Počet pokusov (`n_init=10`)**
  Náhodná inicializácia môže viesť k lokálnym minimám. Spustením algoritmu 10-krát a výberom najlepšieho výsledku podľa celkového `cost` sa eliminuje závislosť od náhody. Už pri 5 pokusoch je výsledok stabilný, 10 pokusov poskytuje takmer 100 % úspešnosť.
- **Maximálny počet iterácií (`max_iters=100`)**
  Algoritmus zvyčajne konverguje do 20–30 iterácií. Hodnota 100 je dostatočne vysoká, aby sa zaručila konvergencia aj v najhoršom prípade.
- **Tolerancia konvergencie (`tol=1e-4`)**
  Ak sa centroidy medzi iteráciami zmenia o menej ako 0,0001, algoritmus sa považuje za skonvergovaný. Táto hodnota je štandardná a zaisťuje vysokú presnosť bez zbytočného predlžovania behu.

3.2 k-medoids

- **Maximálny počet iterácií (`max_iters=30`)**
  k-medoids konverguje pomalšie ako k-means. 30 iterácií je kompromis medzi kvalitou a časom – ďalších 10 iterácií už neprinesie výrazné zlepšenie.
- **Počet kandidátov na medoid v jednom klastri (max. 200)**
  Pre klaster s viac ako 200 bodmi sa testuje náhodný výber 200 kandidátov. Toto obmedzenie znižuje výpočtovú náročnosť (z $O(n^2)$ na $O(n)$) pri zachovaní vysokej pravdepodobnosti nájdenia lepšieho medoidu.

3.3 Divízne zhlukovanie

- **Použitie k-means s $k = 2$ na rozdelenie**
  Každé delenie je binárne. Použitie plného k-means (namiesto náhodného rozdelenia) zaisťuje kvalitné podklastre.

3.4 Spoločné parametre

- **Hranica úspešnosti (500)**
  Priamo podľa zadania. Všetky algoritmy túto hranicu výrazne podliezajú.
- **Verbose=True**
  Tento parameter slúži na výpis priebežných vysledkov algoritmov.
- **Počet výsledných klastrov (`k=20`)**
  Počet výsledných klastrov môžeme nastaviť ľubovolný ale optimalna hodnota bola k=20 čo bude vysvetlené v sekcií Analýza vplyvu počtu klastrov k

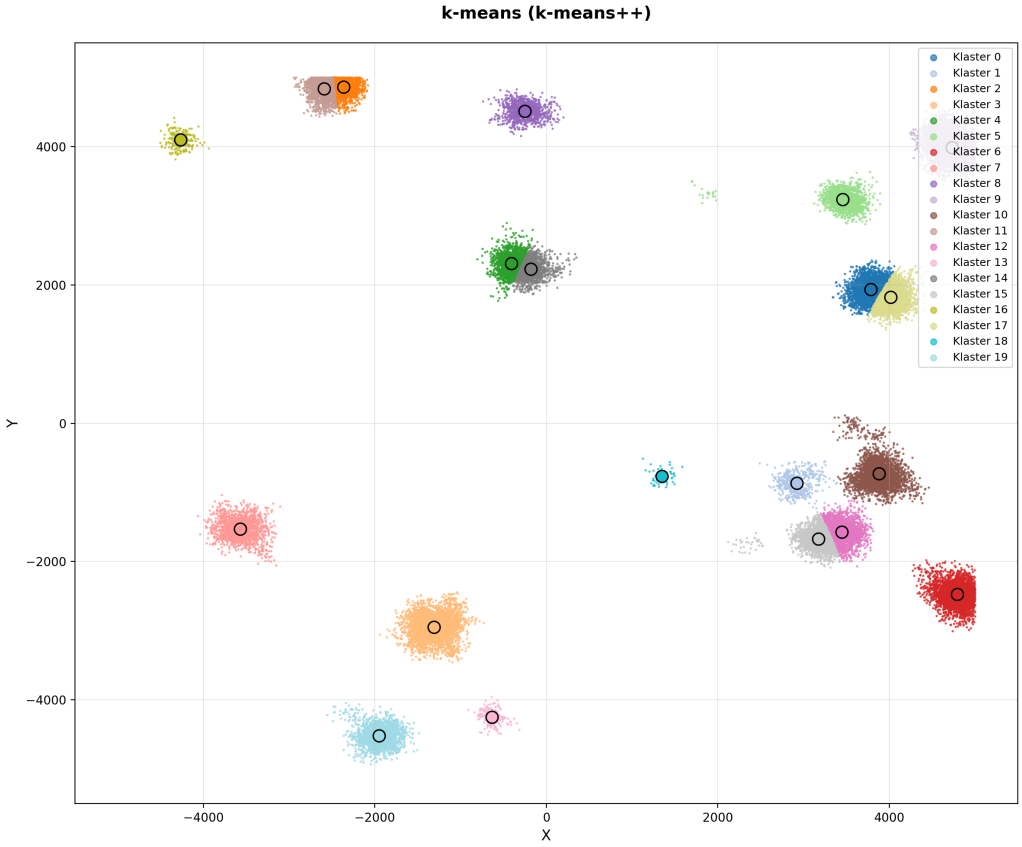
Zvolené parametre predstavujú optimálny kompromis medzi kvalitou zhlukovania, stabilitou výsledkov a výpočtovým časom. Bez k-means++ inicializácie a viacerých pokusov by k-means často zlyhal. Obmedzenie kandidátov v k-medoids bolo nevyhnutné pre beh na bežnom počítači.

4 Vyhodnotenie úspešnosti

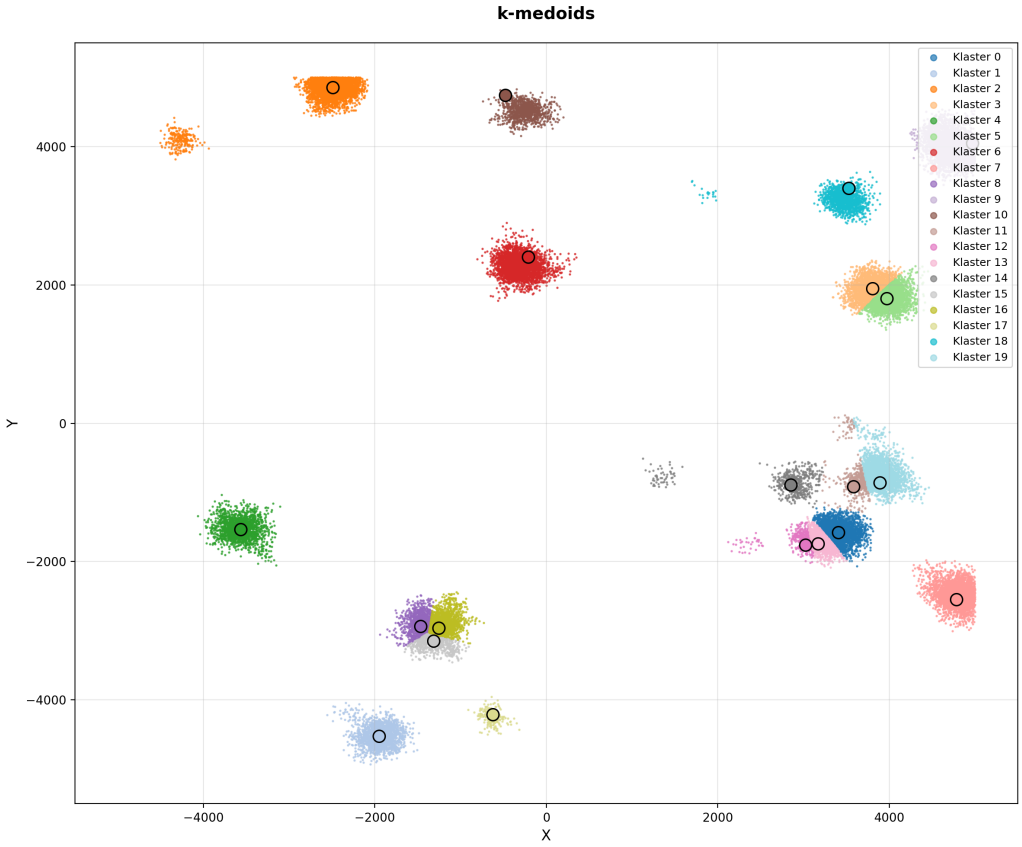
Pre každý klaster bola vypočítaná priemerná euklidovská vzdialenosť jeho bodov od stredu (centroidu alebo medoidu). Podmienka zadania bola splnená, ak žiadny klaster neprekročil hranicu 500. Všetky tri algoritmy túto podmienku splnili.

5 Vizualizácia

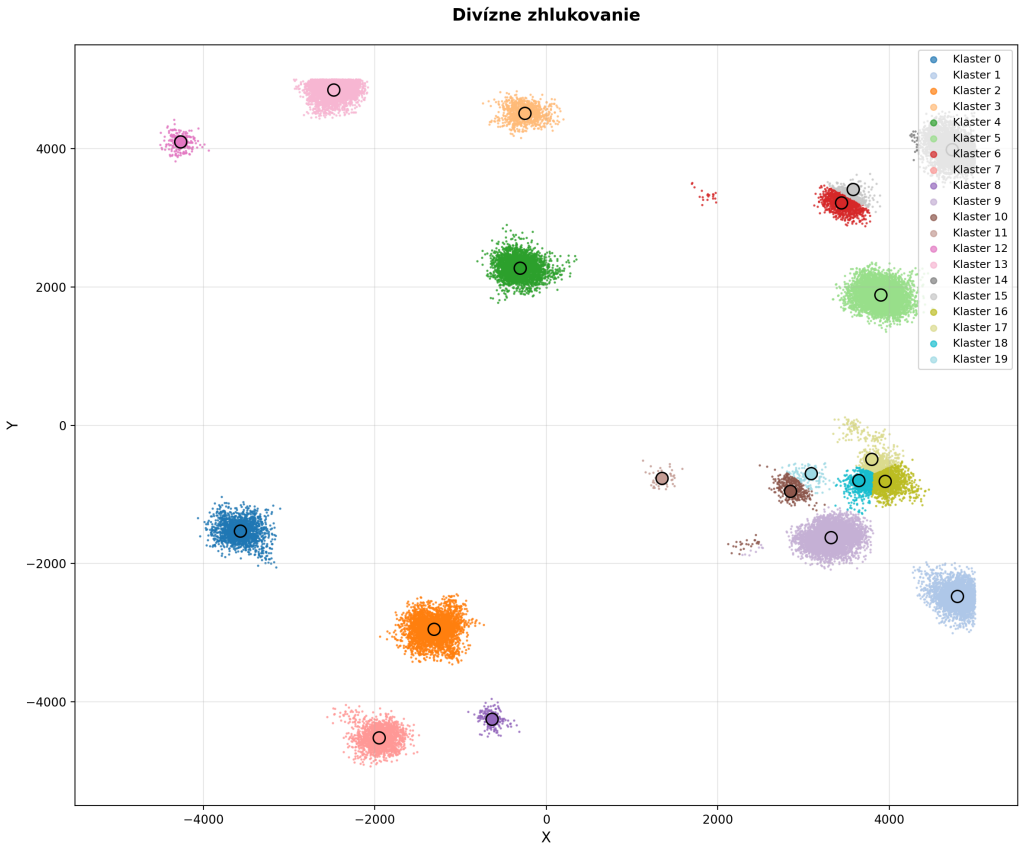
Každý klaster je zobrazený inou farbou. Stredy zhlukov sú zvýraznené farebným kruhom s čiernym okrajom, ktorý má rovnakú farbu ako príslušný klaster. Obrázky boli uložené vo vysokom rozlíšení.



Obr. 1: k-means(centroid) pri počte klasterov k=20



Obr. 2: k-means(medoid) pri počte klasterov k=20



Obr. 3: Divízne zhlukovanie pri počte klasterov k=20

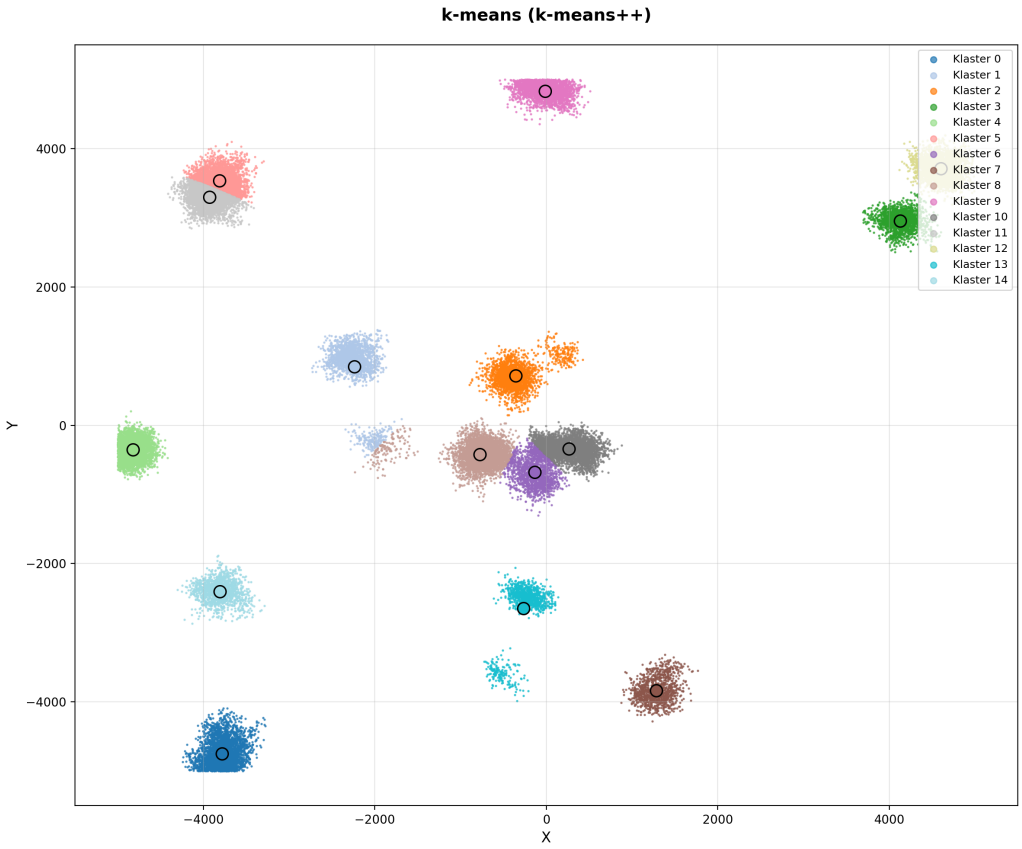
6 Analýza vplyvu počtu klastrov k

Dataset bol generovaný tak, aby obsahoval presne 20 prirodzených zhlukov. Preto je očakávané, že pri $k = 20$ budú výsledky najlepšie. Na overenie tejto hypotézy boli algoritmy spustené aj pri $k = 15$ a $k = 12$.

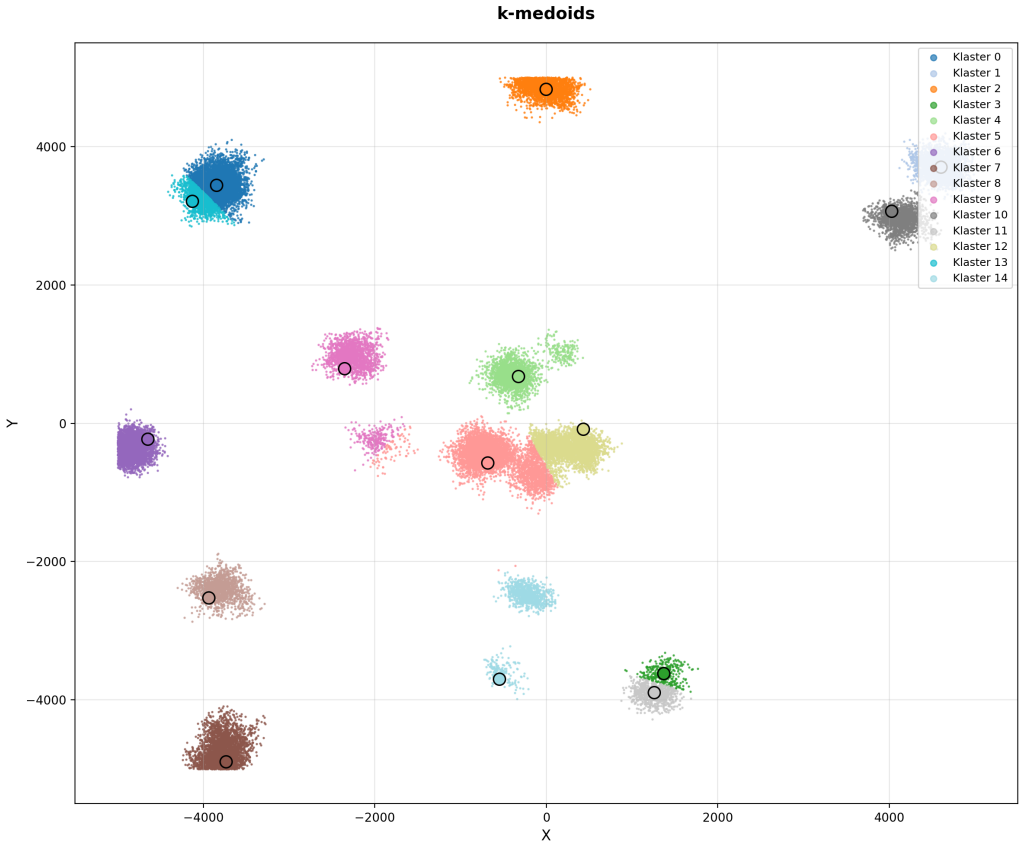
Počet klastrov (k)	**k-means (centroid)**	**k-medoids**	**Divízne**	**Poznámka**
$k = 20$ (správny počet)	**Áno**	**Áno**	**Áno**	Všetky algoritmy uspeli
$k = 15$	**Áno**	**Nie** (1098)	**Áno**	Len k-medoids zlyhal
$k = 12$	**Nie** (609)	**Nie** (1127)	**Nie** (501)	Žiadny algoritmus neuspeli

Vysvetlenie výsledkov:

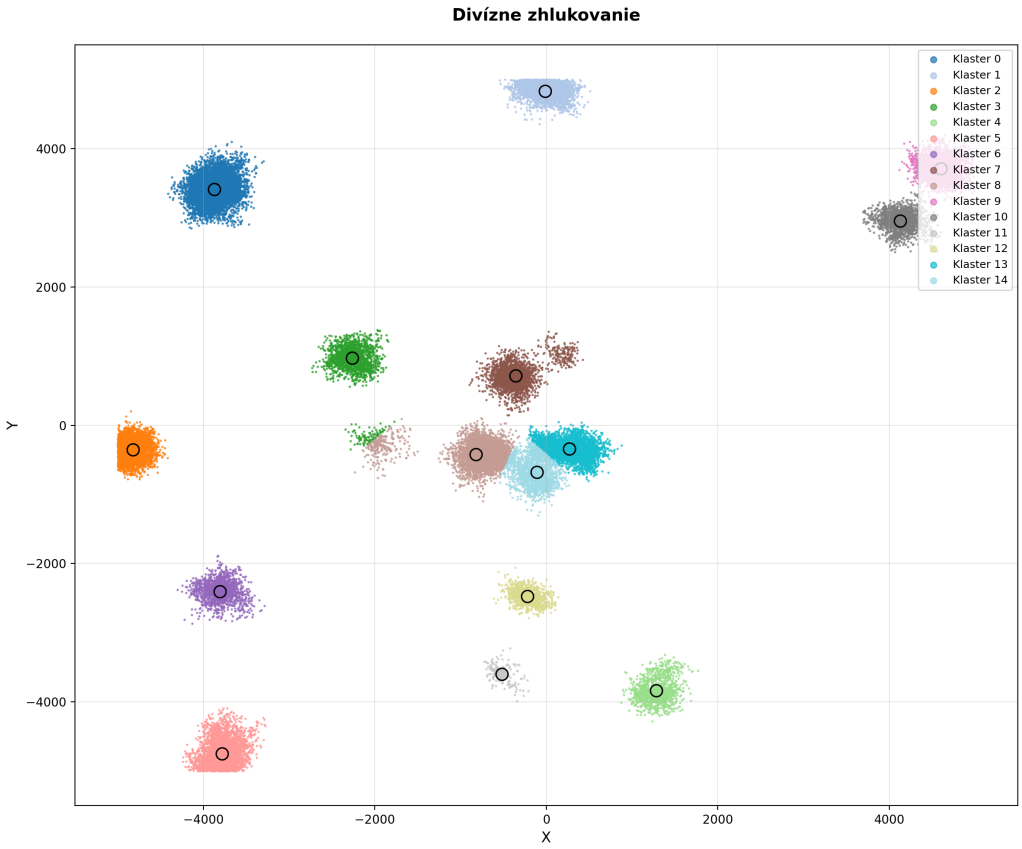
- Pri $k = 20$ sú všetky algoritmy schopné nájsť prirodzenú štruktúru dát. Priemerné vzdialenosti sú výrazne pod hranicou 500.
- Pri $k = 15$ musí algoritmus „spojiť“ 5 párov prirodzených zhlukov. k-means (centroid) to zvládne, pretože centroid sa posunie do stredu spojených zhlukov. k-medoids však musí vybrať jeden skutočný bod ako medoid, ktorý nemôže byť v strede – preto jeden klaster „vybuchne“ na 1098.



Obr. 4: k-means(centroid) pri počte klasterov k=15

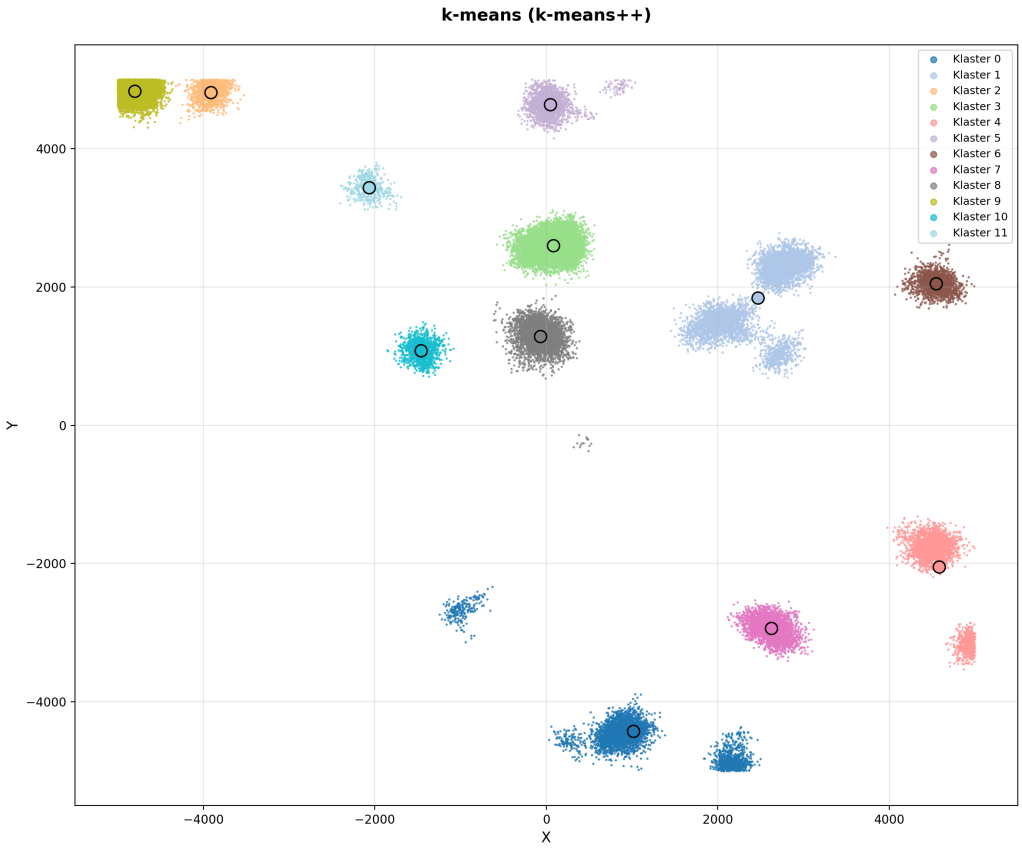


Obr. 5: k-means(medoid) pri počte klasterov k=15 (Zlyhal)

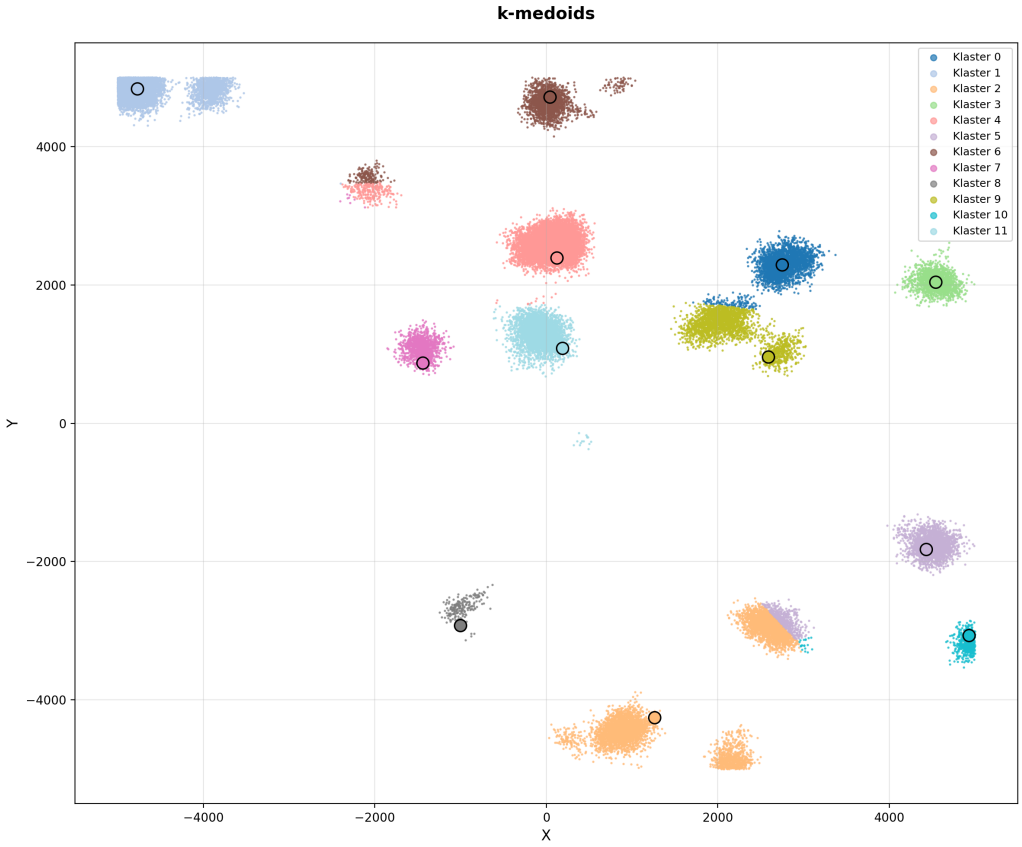


Obr. 6: Divízne zhlukovanie pri počte klasterov k=15

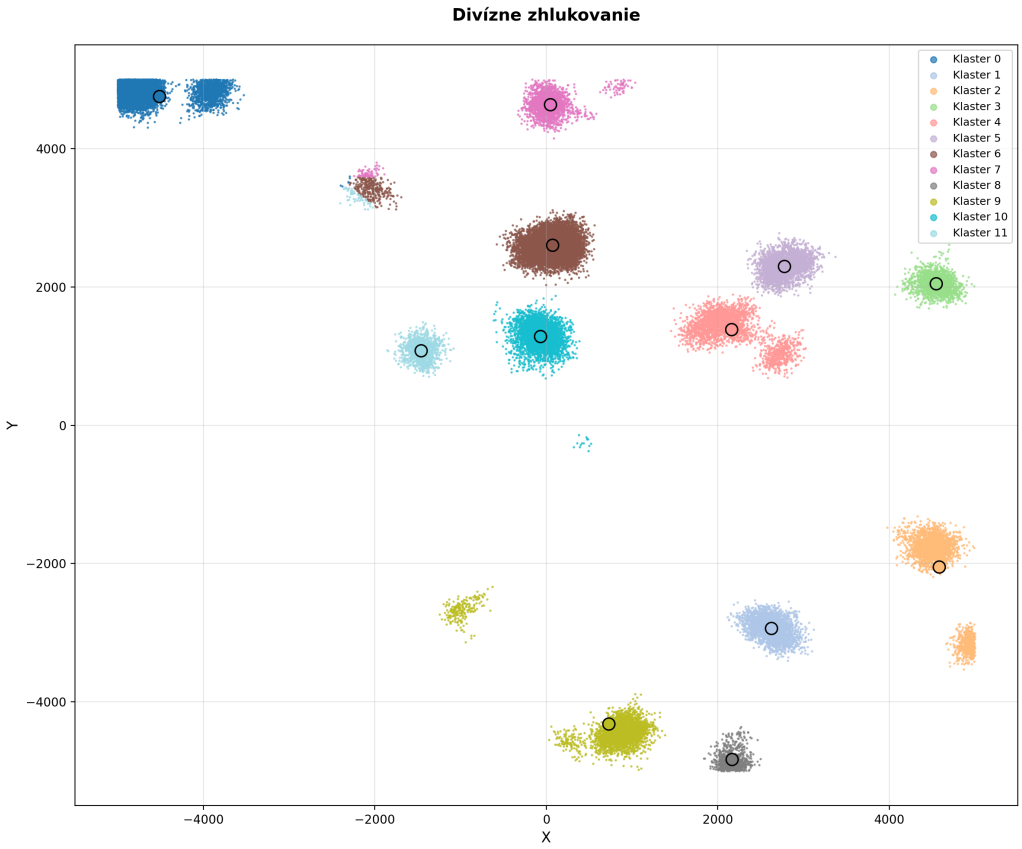
- Pri $k = 12$ je nútených ešte viac spájaní. Všetky algoritmy vytvoria veľké hybridné klastre, ktorých body sú príliš ďaleko od stredu. Najhoršie dopadne k-medoids (1127), najlepšie divízne zhlukovanie (501 ale stále nad limitom).



Obr. 7: k-means(centroid) pri počte klasterov k=12 (Zlyhal)



Obr. 8: k-means(medoid) pri počte klasterov k=15 (Zlyhal)



Obr. 9: Divízne zhlukovanie pri počte klasterov k=12 (Zlyhal)

Záver experimentu: *Počet klastrov k musí zodpovedať skutočnej štruktúre dát. Pri nesprávnom k zlyhávajú aj dobre naprogramované algoritmy. Najcitlivejší na nesprávne k je k-medoids, najodolnejší k-means s centroidmi.*

Tento experiment potvrdzuje dôležitosť správneho odhadu počtu zhlukov pred spustením zhlukovania.

7 Porovnanie algoritmov

Všetky tri algoritmy boli implementované od základov a testované na rovnakom datasete s 40 020 bodmi a $k = 20$ zhlukmi. Pri správnom počte zhlukov dosiahli všetky algoritmy požadovaný výsledok – priemerná vzdialenosť bodov od stredu bola výrazne pod hranicou 500.

Vlastnosť	**k-means (centroid)**	**k-medoids**	**Divízne**
Úspech pri $k = 20$	Áno	Áno	Áno
Úspech pri $k = 15$	Áno	Nie (1098)	Áno
Úspech pri $k = 12$	Nie (609)	Nie (1127)	Nie (501)
Citlivosť na nesprávne k	Nízka	Vysoká	Stredná
Odolnosť voči odľahlým bodom	Nízka	Vysoká	Nízka
Stabilita výsledku	Vysoká	Vysoká	Vysoká
Pocitová rýchlosť	Stredná	Najpomalší	Najrýchlejší
Teoretická zložitosť	$O(nkI)$	$O(nkcI)$	$O(nk \log k)$
Výhody	Rýchly, stabilný	Robustný	Efektívny, hierarchia
Nevýhody	Citlivý na outliery	Pomalý	Nevyvážené delenie

Zhrnutie porovnania:

- **Divízne zhlukovanie** bolo pocitovo najrýchlejšie a prekvapivo efektívne.
- **k-means s centroidmi** s k-means++ inicializáciou je najuniverzálnejší a najstabilnejší.
- **k-medoids** je najrobustnejší voči odľahlým bodom, ale najpomalší a najcitlivejší na zníženie počtu zhlukov.

Pri nesprávnom k ako prvý zlyhá k-medoids, ako posledný k-means s centroidmi. Divízne zhlukovanie je dobrý kompromis medzi rýchlosťou a kvalitou.

8 Záver

Všetky tri algoritmy úspešne vyriešili zadanú úlohu. Najlepšie výsledky z hľadiska rýchlosti a stability dosiahol k-means(centroid) aj divízne zhlukovanie. k-medoids poskytol najrobustnejšie riešenie voči možným odľahlým hodnotám.

Kľúčovým zistením bolo, že náhodná inicializácia k-means nestačí na spoľahlivé vyriešenie zadania. Použitie k-means++ a viacnásobných pokusov bolo nevyhnutné na dosiahnutie stabilného a správneho výsledku.

Projekt bol úspešne dokončený s vlastnou implementáciou všetkých algoritmov, detailným výpisom priebehu, interaktívnym výberom algoritmu a prehľadnou vizualizáciou.